Coding & Solution

Date	30 June 2026
Team ID	
Project Name	Voice-Based Concept Understanding Analyser (VBCUA)
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	Github URL: https://github.com/PudiBhargavi/VBCUA.git Hosted link: https://malfoydraco-vbcua.hf.space
Programming Language(s)	Python
Framework(s) Used	Streamlit, PyTorch (via Whisper), Google Generative AI SDK
Key Features Implemented	Speech-to-text transcription (Whisper), semantic similarity scoring (Sentence-BERT), audio fluency analysis (Librosa — filler words, pause ratio, RMS energy), combined scoring engine, AI-generated feedback (Gemini), waveform visualization, downloadable PDF report
Pending / Incomplete Features	None — all planned core features implemented
Setup / Run Instructions	1. Create virtual environment: python -m venv venv 2. Activate it and install dependencies: pip install -r requirements.txt 3. Add Gemini API key to .env file 4. Run: streamlit run app.py

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The application is structured into independent modules (transcription, semantic analysis, audio features, scoring, feedback generation, report generation), each independently testable, then integrated via a single pipeline function called from the Streamlit UI.